



HOA SEN
UNIVERSITY

Lecture 14

Algorithms

- Reference: Chapter 15

Designing A Set Of Relations

- The Approach of Relational Synthesis (Bottom-up Design):
 - Assumes that all possible functional dependencies are known.
 - First constructs a minimal set of FDs
 - Then applies algorithms that construct a target set of 3NF or BCNF relations.
 - Additional criteria may be needed to ensure the the *set of relations* in a relational database are satisfactory (see Algorithms 15.4 and 15.6).

Designing A Set Of Relations (2)

- Goals:
 - Lossless join property (a must)
 - Want to be able to reconstruct big (e.g. universal) relation by joining smaller ones (using natural joins)
(i.e. $R_1 \bowtie R_2 = R$)
 - Algorithm 11.1 tests for general losslessness.
 - Dependency preservation property
 - Want to minimize the cost of global integrity constraints based on FD's (i.e. avoid big joins in assertions)
 - Algorithm 15.3 decomposes a relation into BCNF components by sacrificing the dependency preservation.
 - Redundancy Avoidance
 - Avoid unnecessary data duplication (the motivation for decomposition)

Ensuring Lossless Joins

- A decomposition of R : $R = R_1 \cup R_2$

Is lossless if

$$R_1 \cap R_2 \rightarrow R_1, \text{ or}$$

$$R_1 \cap R_2 \rightarrow R_2$$

(i.e., intersecting attributes must form a superkey for one of the resulting smaller relations)

Decomposition Goal #2: Dependency preservation

- Goal: efficient integrity checks of FD's

An example w/ no DP:

$R = (\text{bname}, \text{bcity}, \text{assets}, \text{cname}, \text{lno}, \text{amt})$

$\text{bname} \rightarrow \text{bcity}, \text{assets}, \text{lno}$

$\text{lno} \rightarrow \text{amt}, \text{bname}$

- Decomposition: $R = R_1 \cup R_2$

$R_1 = (\text{bname}, \text{assets}, \text{cname}, \text{lno})$

$R_2 = (\text{lno}, \text{bcity}, \text{amt})$

- Lossless but not DP. Why?

- Ans: $\text{bname} \rightarrow \text{bcity}, \text{assets}$ crosses 2 tables

Decomposition Goal #2: Dependency preservation

- To ensure best possible efficiency of FD checks ensure that only a SINGLE table is needed in order to check each FD

i.e. ensure that: $A_1 A_2 \dots A_n \rightarrow B_1 B_2 \dots B_m$

Can be checked by examining

$R_i = (\dots, A_1, A_2, \dots, A_n, \dots, B_1, \dots, B_m, \dots)$

- To test if the decomposition $R = R_1 \cup R_2 \cup \dots \cup R_n$ is Dependency preservation
 - (1) see which FD's of R are covered by $R_1, R_2, \dots, R_n$
 - (2) compare the closure of (1) with the closure of FD's of R

Decomposition Goal #2: Dependency preservation

- Example: Give $R(A, B, C, D)$
and $F = \{A \rightarrow B, AB \rightarrow D, C \rightarrow D\}$
consider $R = R_1 \cup R_2$
 $R_1 = (A, B, D)$, $R_2 = (C, D)$

(1) $F^+ = \{A \rightarrow BD, C \rightarrow D\}^+$

(2) $G^+ = \{A \rightarrow BD, C \rightarrow D, \dots\}^+$

(3) $F^+ = G^+$

Note: G^+ cannot introduce new FDs not in F^+
Decomposition is Dependency Preservation

Relational Decompositions

- Relation Decomposition and Insufficiency of Normal Forms:
 - Universal Relation Schema:
 - A relation schema $R = \{A_1, A_2, \dots, A_n\}$ that includes all the attributes of the database.
 - Universal relation assumption:
 - Every attribute name is unique.

Relational Decompositions (2)

- Relation Decomposition and Insufficiency of Normal Forms (cont.):
 - Decomposition:
 - The process of decomposing the universal relation schema R into a set of relation schemas $D = \{R_1, R_2, \dots, R_m\}$ that will become the relational database schema by using the functional dependencies.
 - Attribute preservation condition:
 - Each attribute in R will appear in at least one relation schema R_i in the decomposition so that no attributes are “lost”.

Insufficiency of Normal Form

- Another goal of decomposition is to have each individual relation R_i in the decomposition D be in BCNF or 3NF.

Example:

EMP_LOCS(ENAME, PLOCATION) $\in$ 3NF and BCNF

(Any relation schema with only two attributes is automatically in BCNF)

It still generates spurious tuples even joining with a BCNF relation
PROJECT(PNAME, PNUMBER, PLOCATION, DNUM)

- Additional properties of decomposition are needed to prevent from generating spurious tuples

Definition of Projection

- Dependency Preservation Property of a Decomposition:
 - Definition: Given a set of dependencies F on R , the **projection** of F on R_i , denoted by $\Pi_{R_i}(F)$ where R_i is a subset of R , is the set of dependencies $X \rightarrow Y$ in F^+ such that the attributes in $X \cup Y$ are all contained in R_i .
 - Hence, the projection of F on each relation schema R_i in the decomposition $D = \{R_1, R_2, \dots, R_m\}$ is the set of functional dependencies in F^+ , the closure of F , such that all their left- and right-hand-side attributes are in R_i .

Properties of Relational Decompositions

- Dependency Preservation Property:
 - A decomposition $D = \{R_1, R_2, \dots, R_m\}$ of R is **dependency-preserving** with respect to F if the union of the projections of F on each R_i in D is equivalent to F ; that is
$$((\pi_{R_1}(F)) \cup \dots \cup (\pi_{R_m}(F)))^+ = F^+$$

Properties of Relational Decompositions (2)

- Claim 1:
 - It is always possible to find a dependency-preserving decomposition D with respect to F such that each relation R_i in D is in 3NF.
(Algorithm 15.2 creates a dependency-preserving decomposition $D = \{R_1, R_2, \dots, R_m\}$ of universal relation R based on a set of functional dependencies F such that each R_i in D is in 3NF)
- If a decomposition is not dependency-preserving, then some dependencies are **lost** in the decomposition

Properties of Relational Decompositions (3)

- Lossless (Non-additive) Join Property of a Decomposition:

- Definition: Lossless join property:

a decomposition $D = \{R_1, R_2, \dots, R_m\}$ of R has the **lossless (nonadditive) join property** with respect to the set of dependencies F on R if, for every relation state r of R that satisfies F , the following holds, where $*$ is the NATURAL JOIN of all the relations in D :

$$* (\pi_{R_1}(r), \dots, \pi_{R_m}(r)) = r$$

- Note: The word loss in lossless refers to loss of information, not to loss of tuples. In fact, for “loss of information” a better term is “**addition of spurious information**”

Algorithm 15.3: Testing for Lossless Join Property

- **Input:** A universal relation R , a decomposition $D = \{R_1, R_2, \dots, R_m\}$ of R , and a set F of functional dependencies.
 1. Create an initial matrix S with one row i for each relation R_i in D , and one column j for each attribute A_j in R .
 2. Set $S(i,j) := b_{ij}$ for all matrix entries. (* each b_{ij} is a distinct symbol associated with indices (i,j) *).
 3. For each row i representing relation schema R_i
 {for each column j representing attribute A_j
 {if (relation R_i includes attribute A_j) then set $S(i,j) := a_j$ };};
- (* each a_j is a distinct symbol associated with index (j) *)

Algorithm 15.3 (2)

4. Repeat the following loop until a complete loop execution results in no changes to S
 - {for each functional dependency $X \rightarrow Y$ in F
 - {for all rows in S *which have the same symbols* in the columns corresponding to attributes in X
 - {make the symbols in each column that correspond to an attribute in Y be the same in all these rows as follows:
 - If any of the rows has an “a” symbol for the column, set the other rows to that *same* “a” symbol in the column.
 - If no “a” symbol exists for the attribute in any of the rows, choose one of the “b” symbols that appear in one of the rows for the attribute and set the other rows to that same “b” symbol in the column ;};
 - };
5. If a row is made up entirely of “a” symbols, then the decomposition has the lossless join property; otherwise it does not.

Example

- (a) $R = \{SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS\}$ $D = \{R_1, R_2\}$
 $R_1 = EMP_LOCS = \{ENAME, PLOCATION\}$
 $R_2 = EMP_PROJ1 = \{SSN, PNUMBER, HOURS, PNAME, PLOCATION\}$
 $F = \{SSN \rightarrow ENAME; PNUMBER \rightarrow \{PNAME, PLOCATION\}; \{SSN, PNUMBER\} \rightarrow HOURS\}$

	SSN	ENAME	PNUMBER	PNAME	PLOCATION	HOURS
R_1	b_{11}	a_2	b_{13}	b_{14}	a_5	b_{16}
R_2	a_1	b_{22}	a_3	a_4	a_5	a_6



(no changes to matrix after applying functional dependencies)

Example

(b)

EMP		PROJECT			WORKS_ON		
SSN	ENAME	PNUMBER	PNAME	PLOCATION	SSN	PNUMBER	HOURS

- (c) $R = \{SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS\}$ $D = \{R_1, R_2, R_3\}$
 $R_1 = EMP = \{SSN, ENAME\}$
 $R_2 = PROJ = \{PNUMBER, PNAME, PLOCATION\}$
 $R_3 = WORKS_ON = \{SSN, PNUMBER, HOURS\}$
 $F = \{SSN \rightarrow \{ENAME\}; PNUMBER \rightarrow \{PNAME, PLOCATION\}; \{SSN, PNUMBER\} \rightarrow HOURS\}$

	SSN	ENAME	PNUMBER	PNAME	PLOCATION	HOURS
R_1	a_1	a_2	b_{13}	b_{14}	b_{15}	b_{16}
R_2	b_{21}	b_{22}	a_3	a_4	a_5	b_{26}
R_3	a_1	b_{32}	a_3	b_{34}	b_{35}	a_6

(original matrix S at start of algorithm)

	SSN	ENAME	PNUMBER	PNAME	PLOCATION	HOURS
R_1	a_1	a_2	b_{13}	b_{14}	b_{15}	b_{16}
R_2	b_{21}	b_{22}	a_3	a_4	a_5	b_{26}
R_3	a_1	b_{32} a_2	a_3	b_{34} a_4	b_{35} a_5	a_6

(matrix S after applying the first two functional dependencies - last row is all "a" symbols, so we stop)

Testing Binary Decompositions

- Testing Binary Decompositions for Lossless Join Property
 - **Binary Decomposition:** Decomposition of a relation R into two relations.
 - **PROPERTY NJB (lossless join test for binary decompositions):**
A decomposition $D = \{R_1, R_2\}$ of R has the lossless join property with respect to a set of functional dependencies F on R *if and only if* either
 - The FD $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$ is in F^+ , or
 - The FD $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$ is in F^+ .

Binary Decomposition Example

$$ED1 \cap ED2 = \{Dnumber\}$$

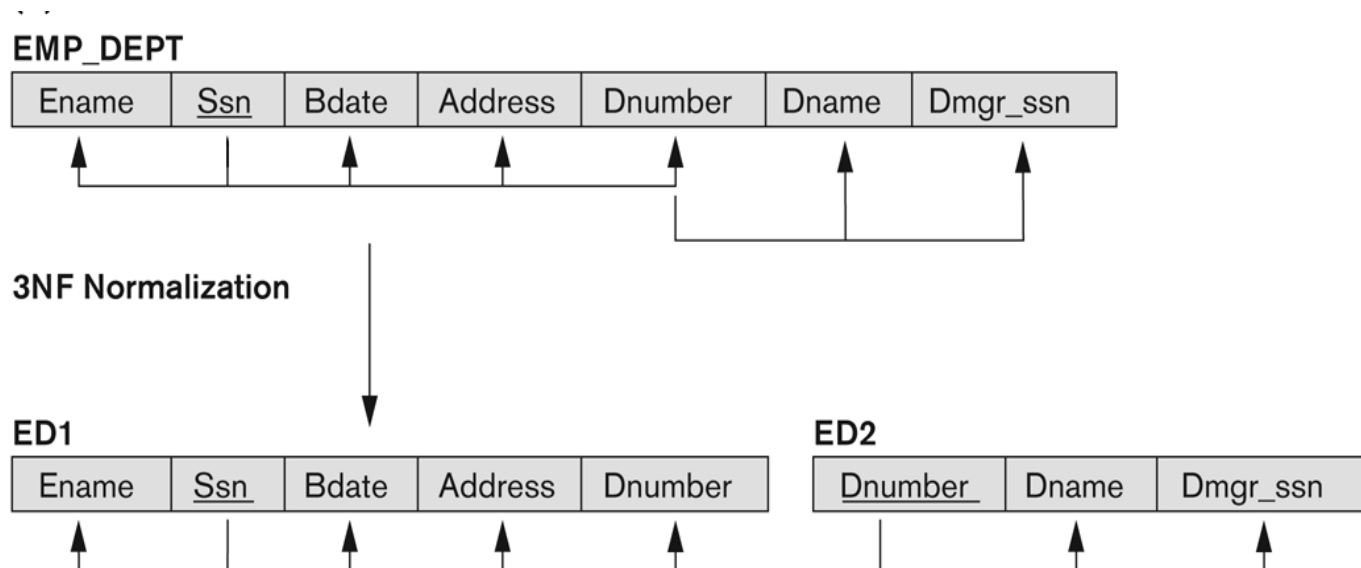
$$ED2 - ED1 = \{Dname, Dmgr_ssn\}$$

And

$$ED1 \cap ED2 \rightarrow ED2 - ED1, \text{ or}$$

$$Dnumber \rightarrow \{Dname, Dmgr_ssn\}$$

So, the binary decomposition is **non-additive**



- **Claim 2 (Preservation of non-additivity in successive decompositions):**
 - If a decomposition $D = \{R_1, R_2, \dots, R_m\}$ of R has the lossless (non-additive) join property with respect to a set of functional dependencies F on R ,
 - and if a decomposition $D_i = \{Q_1, Q_2, \dots, Q_k\}$ of R_i has the lossless (non-additive) join property with respect to the projection of F on R_i ,
 - then the decomposition $D_2 = \{R_1, R_2, \dots, R_{i-1}, Q_1, Q_2, \dots, Q_k, R_{i+1}, \dots, R_m\}$ of R has the non-additive join property with respect to F .

Algorithms 15.5 - Relational Decomposition into BCNF

- Algorithm 15.5: Relational Decomposition into BCNF with Lossless (non-additive) join property
 - Input: A universal relation R and a set of functional dependencies F on the attributes of R .
 1. Set $D := \{R\}$;
 2. While there is a relation schema Q in D that is not in BCNF do
 - {
 - choose a relation schema Q in D that is not in BCNF;
 - find a functional dependency $X \rightarrow Y$ in Q that violates BCNF;
 - replace Q in D by two relation schemas $(Q - Y)$ and $(X \cup Y)$;
 - };

Assumption: No null values are allowed for the join attributes.

Algorithms 15.4 - Relational Synthesis Algorithm

Algorithm 15.4: Relational Synthesis into 3NF with Dependency Preservation (Relational Synthesis Algorithm)

- Input: A universal relation R and a set of functional dependencies F on the attributes of R.
- 1. Find a **minimal cover** G for F (use Algorithm 14.2);
- 2. For each **left-hand-side** X of a functional dependency that appears in G,
 create a relation schema in D with attributes $\{X \cup \{A_1\} \cup \{A_2\} \dots \cup \{A_k\}\}$,
 where $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_k$ are the only dependencies in G with X as left-hand-side (X is the key of this relation) ;
- 3. Place any remaining attributes (that have not been placed in any relation) in a single relation schema to ensure the attribute preservation property.

Example – Relation Synthesis Algorithm

- $R(\text{Emp_SSN}, \text{PNo}, \text{ESal}, \text{EPhone}, \text{DNo}, \text{PName}, \text{PLocation})$
- $F = \{ F_1: \text{Emp_SSN} \rightarrow \text{ESal}, \text{EPhone}, \text{DNo},$
 $F_2: \text{PNo} \rightarrow \text{PName}, \text{PLocation},$
 $F_3: \text{Emp_SSN}, \text{PNo} \rightarrow \text{ESal}, \text{EPhone}, \text{DNo}, \text{PName},$
 $\text{PLocation})$
- ➔ Key of the universal relation: $\{\text{Emp_SSN}, \text{PNo}\}$

Example – Relation Synthesis Algorithm (2)

- Step1: Find a minimal cover G:
 $\{F_1: \text{Emp_SSN} \rightarrow \text{ESal}, \text{EPhone}, \text{DNo},$
 $F_2: \text{PNo} \rightarrow \text{PName}, \text{PLocation} \}$
- Step2: Create a relation schema based on left-hand side X of a functional dependency
R1(Emp SSN, ESal, EPhone, DNo)
R2(PNo, PName, PLocation)

Comparison of BCNF and 3NF

- It is always possible to decompose a relation into relations in 3NF and
 - the decomposition is lossless
 - the dependencies are preserved
- It is always possible to decompose a relation into relations in BCNF and
 - the decomposition is lossless
 - it may not be possible to preserve dependencies

Problems with Null Values

- One problem of having null values for a foreign key attribute may cause loss of tuples in a JOIN operation (Example in next two slides)
- A related problem is that, in decomposing a relation with null values, if the nulls are not appropriately reserved, an inner join operation will result in loss of tuples. Dangling tuples refer to the tuples in danger in the join

Null Values in Attribute Dnum

(a)

EMPLOYEE

Ename	<u>Ssn</u>	Bdate	Address	Dnum
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1
Berger, Anders C.	999775555	1965-04-26	6530 Braes, Bellaire, TX	NULL
Benitez, Carlos M.	888664444	1963-01-09	7654 Beech, Houston, TX	NULL

Issues with NULL-value joins. (a) Some EMPLOYEE tuples have NULL for the join attribute Dnum.

DEPARTMENT

Dname	<u>Dnum</u>	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

Join Operation with Relation of Nulls

(b) Missing tuples in a NATURAL JOIN

Ename	Ssn	Bdate	Address	Dnum	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

(b) Result of applying NATURAL JOIN to the EMPLOYEE and DEPARTMENT relations.

LEFT OUTER JOIN must be used

(c)

Ename	Ssn	Bdate	Address	Dnum	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555
Berger, Anders C.	999775555	1965-04-26	6530 Braes, Bellaire, TX	NULL	NULL	NULL
Benitez, Carlos M.	888665555	1963-01-09	7654 Beech, Houston, TX	NULL	NULL	NULL

(c) Result of applying LEFT OUTER JOIN to the EMPLOYEE and DEPARTMENT.

Example for Dangling Tuples

The dangling tuple problem.

(a) The relation EMPLOYEE_1 (includes all attributes of EMPLOYEE except Dnum).

EMPLOYEE

Ename	<u>Ssn</u>	Bdate	Address	Dnum
-------	------------	-------	---------	------

(a) EMPLOYEE_1

Ename	<u>Ssn</u>	Bdate	Address
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX
Berger, Anders C.	999775555	1965-04-26	6530 Braes, Bellaire, TX
Benitez, Carlos M.	888665555	1963-01-09	7654 Beech, Houston, TX

Example for Dangling Tuples (2)

(b) The relation EMPLOYEE_2 (includes Dnum attribute with NULL values).

(c) The relation EMPLOYEE_3 (includes Dnum attribute but does not include tuples for which Dnum has NULL values).

(b) EMPLOYEE_2

<u>Ssn</u>	Dnum
123456789	5
333445555	5
999887777	4
987654321	4
666884444	5
453453453	5
987987987	4
888665555	1
999775555	NULL
888664444	NULL

(c) EMPLOYEE_3

<u>Ssn</u>	Dnum
123456789	5
333445555	5
999887777	4
987654321	4
666884444	5
453453453	5
987987987	4
888665555	1

Q & A

